

12 — VocX & STT (voice capture)

Audience: engineers on the voice path; operators sizing the box; anyone asked "why did my recording fail?" **Companion docs:** [04 Running flows](#) · [03 Module interaction](#) · [13 Operations](#) **Code:**

`services/vocx/`, `services/stt/`, `services/atlas/ui/src/components/vocx/`

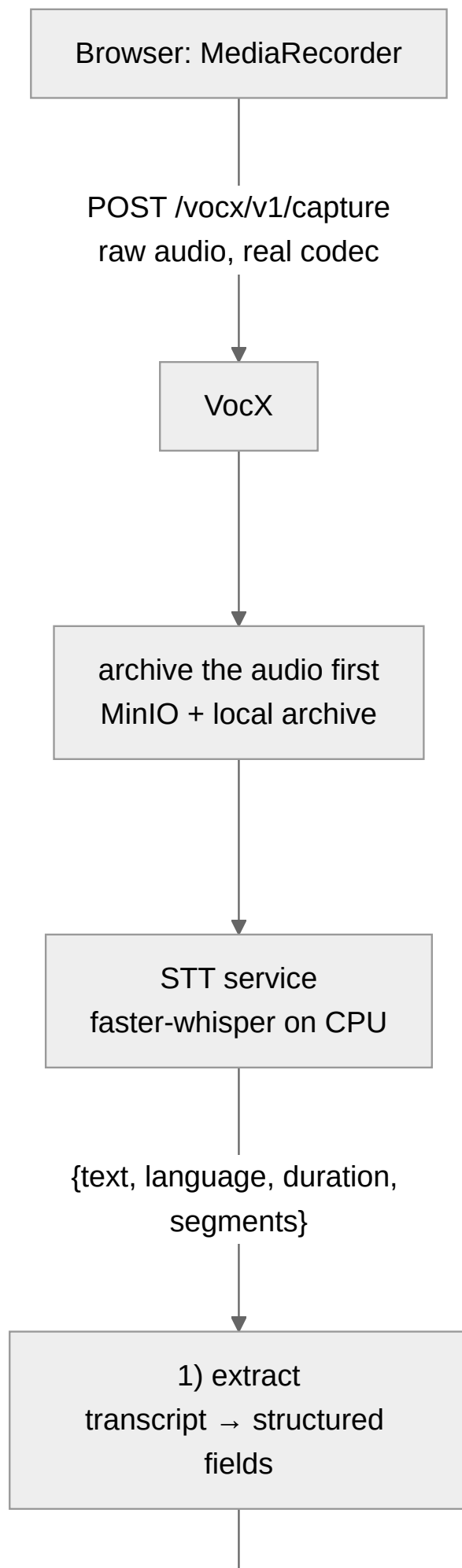
1. What VocX is for

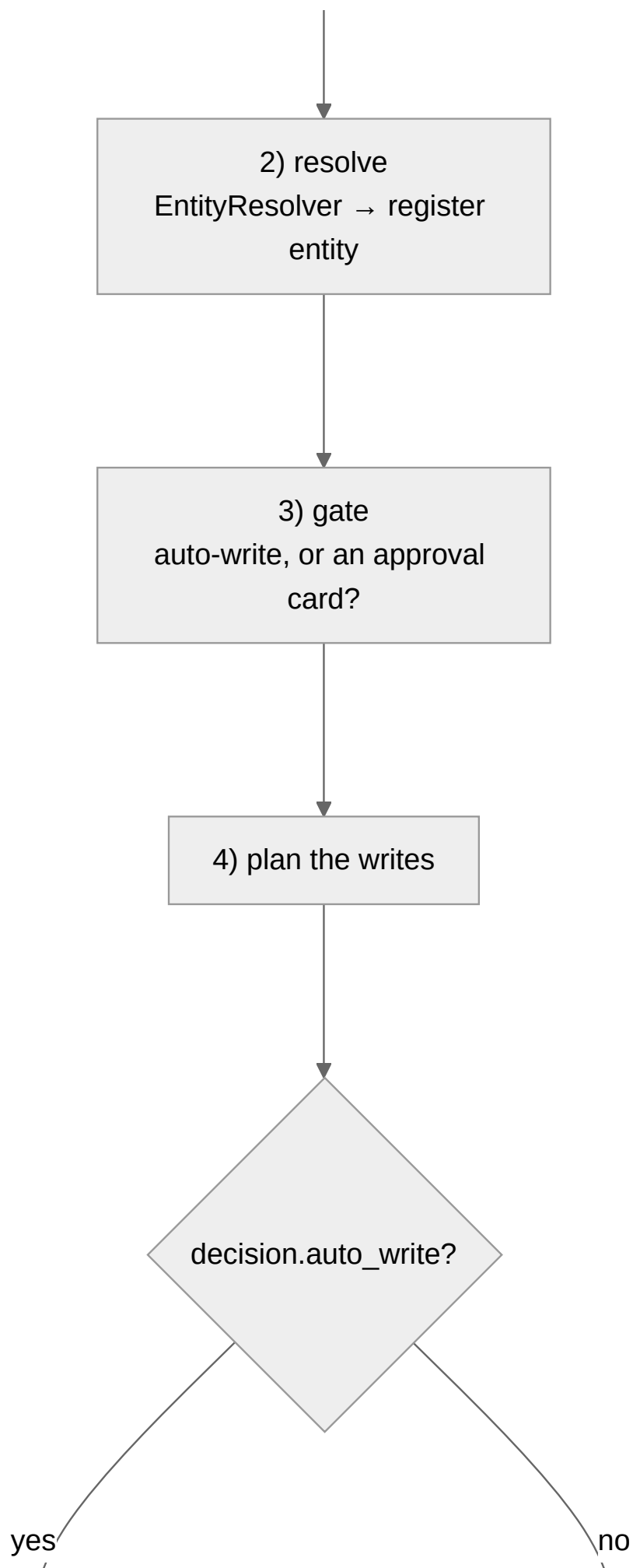
An RM finishes a site visit, opens PRISM on their phone, presses record and talks for two minutes. By the time they are back in the car, the company is matched, an interaction is on the timeline, a lead exists if one did not, and the follow-up is scheduled.

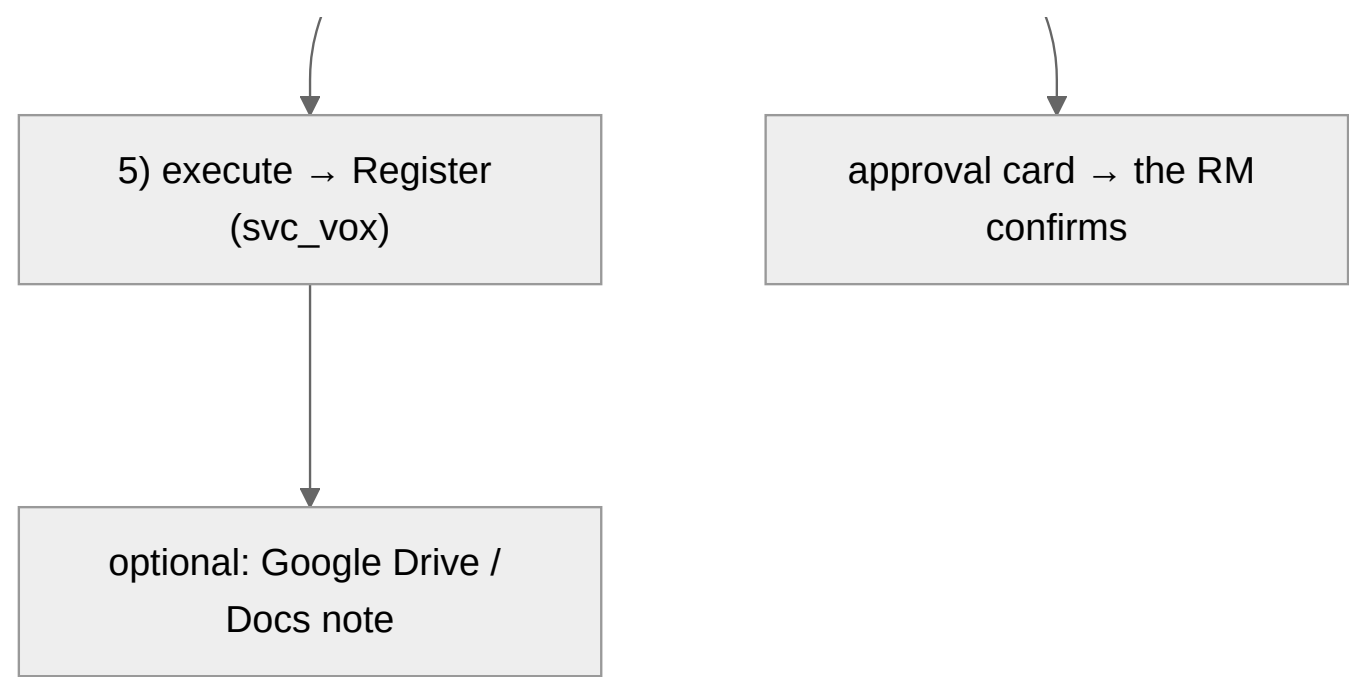
That is the whole product claim, and everything below is in service of two constraints it creates:

1. **The user must never lose a recording they have already made.** Once the audio exists, losing it is unacceptable — so it is archived *before* anything else can fail.
 2. **Transcription accuracy cannot be traded away for speed.** The transcript feeds a credit file.
-

2. The pipeline







`services/vocx/app/vocx/core/pipeline.py::process_capture` is the whole thing in one function. The capture-side facts (`language`, `gps_lat`, `gps_lng`, `location`) ride on `_meta` into the interaction's structured columns and are, in the code's own words, **"never trusted for routing — only recorded."**

English at rest

Whisper is run with `task="translate"`, which makes it emit **English text for any spoken language** (identity for English input). The detected original language still comes back and lands in the interaction's `language` column. So a conversation in Hindi or Gujarati becomes an English record with the original language on the row.

3. Recording length

`services/atlas/ui/src/components/vocx/useRecorder.ts`:

```
const _envCap = Number(import.meta.env.VITE_VOCX_MAX_SECONDS);
export const MAX_SECONDS = Number.isFinite(_envCap) && _envCap > 0
  ? Math.min(600, Math.max(30, Math.round(_envCap)))
  : 180;
```

Property	Value
Default	180 s (3 minutes)
Configurable by	<code>VITE_VOCX_MAX_SECONDS</code> — build-time , via <code>deploy/ui-image/Dockerfile</code>
Clamped to	30–600 s
At the cap	The recorder auto-stops and keeps the clip so far

Nothing is discarded at the cap — a one-second timer checks `secondsRef.current >= MAX_SECONDS` and calls `rec.stop()`. The user gets the recording they made up to that point, transcribed normally.

Changing it

```
# deploy/ui-image/Dockerfile
ARG VITE_VOCX_MAX_SECONDS="180"
ENV VITE_VOCX_MAX_SECONDS=$VITE_VOCX_MAX_SECONDS
```

```
docker compose -f deploy/compose/docker-compose.yml build ui
docker compose -f deploy/compose/docker-compose.yml up -d ui
docker compose -f deploy/compose/docker-compose.yml exec nginx nginx -s reload # ← don't skip
```

Then hard-refresh the browser — the bundle is cached.

Raising the cap beyond ~180 s means longer synchronous decodes. Check the timeout chain in §4 before going much past 300 s, and remember that transcription time scales with clip length **and** with how many captures are running at once.

4. The timeout chain

Four budgets on one request. They only work if they are ordered.

Hop	Budget	Set in
Browser (<code>CAPTURE_TIMEOUT_MS</code>)	300 s	<code>services/atlas/ui/src/api/vocxClient.ts</code>
nginx <code>/vocx/v1/capture</code>	305 s	<code>deploy/nginx/nginx.conf</code>
gateway <code>_SLOW_PATHS</code>	600 s	<code>services/gateway/app/main.py</code>
VocX → STT total budget	240 s	<code>stt.api.budget_s</code>

The browser is deliberately the shortest. The user's own client gives up first, with a message it can explain, rather than the edge returning an opaque 504.

For Helm, the equivalent of the nginx line is `gateway.ingress.slowPaths` on the second Ingress (`ingress-slow.yaml`) — annotations are per-Ingress, not per-path.

The STT retry budget

```
services/vocx/app/vocx/speech/stt.py:
```

```
deadline = time.monotonic() + self.budget_s
for attempt in range(self.attempts):
    remaining = deadline - time.monotonic()
    if attempt and remaining < self.MIN_ATTEMPT_S: # MIN_ATTEMPT_S = 15.0
        break                                     # the FIRST attempt is ALWAYS made
    remaining = max(remaining, 1.0)
    resp = httpx.post(..., timeout=min(float(self.timeout), remaining))
```

Three properties:

- The **total** budget is bounded, not just each attempt. A per-attempt timeout with three retries can silently take three times as long as anyone intended.

- **The first attempt is always made**, even if the budget is nearly gone. A capture is not abandoned without trying.
- A retry is only started if at least 15 s remain — starting one that cannot finish just burns the remaining budget.

On exhaustion, `SttTimeoutError` → VocX answers `504 {"ok": false, "error": ..., "ref": ref}`, and the UI says: *"VocX did not answer in time. The recording is safe — try again in a moment."* Which is true: the audio was archived before transcription started.

5. The stage strip

The UI shows `uploading → transcribing → structuring → done`. Getting this honest took care, because the upload and the transcription have very different shapes.

`services/atlas/ui/src/components/vocx/RecordTab.tsx`:

```
let heard = false; let uploaded = false; let quietPolls = 0;
const uploadCeiling = setTimeout(() => { uploaded = true; }, 60_000);
pollRef.current = setInterval(() => {
  void vocxService.captureStatus(cid).then((s) => {
    if (s !== 'unknown') { heard = true; setStage(s); return; }
    if (!heard && uploaded && ++quietPolls >= 12) { setStage(''); stopPoll(); }
  });
}, 700);
const r = await vocxService.captureAudio(blob, rm, gps, cid, (pct) => {
  if (!heard) setSent(pct);
  if (pct >= 100) uploaded = true;
});
```

The rule: **the patience window starts when the upload finishes, not when it starts**. On a slow mobile connection the upload itself can take a minute, and a server that has not yet *received* the clip cannot be reporting a stage for it. Counting "quiet" polls during the upload made the strip give up on captures that were succeeding.

Upload progress is surfaced as a percentage on the `uploading` chip via axios `onUploadProgress`, wired through `postAudio(..., onUpload)`.

6. The STT service

`services/stt/` — a dedicated container so that **one shared faster-whisper model serves every capture** rather than each VocX worker loading its own.

```
# services/stt/app/config.py
model_size: str = "small"
device: str = "cpu"
compute_type: str = "int8"
beam_size: int = 5
vad_filter: bool = True
cpu_threads: int = 0
model_dir: str = "/opt/models"
preload: bool = True # readiness gates on the model being loaded
max_audio_bytes: int = 25 * 1024 * 1024 # matches VocX's inbound cap
```

The model is baked into the image at build time and the container runs with `HF_HUB_OFFLINE=1`. Serving never depends on huggingface.co, and a `STT_MODEL_SIZE` that is not in `model_dir` **fails fast at startup** instead of quietly downloading half a gigabyte in production.

Decoding is **serialised per process**: ctranslate2 inference is not guaranteed thread-safe on one model instance, and concurrent CPU decodes would thrash anyway.

STT_CPU_THREADS — the setting people get wrong

```
# CPU parallelism for the decoder. 0 hands the choice to CTranslate2, which reads the
# HOST's core count and knows nothing about the container's CPU quota – so a container
# limited to 2 cores can spawn a thread per host core, and those threads then fight
# each other for the 2 it actually has and decode SLOWER than 2 threads would.
cpu_threads: int = 0
```

Value	Behaviour
0	CTranslate2 chooses, based on the host's core count
N	Exactly N decoder threads

Accuracy is untouched by this setting — only scheduling. The model, the beam width and the transcript are identical.

Deployment	CPU limit on <code>stt</code>	Correct value
Helm	<code>limits.cpu: 2</code>	<code>2</code> — match the limit
Compose	<code>none</code>	Derive from <code>nproc</code> ; <code>0</code> is often right, and <code>2</code> on a 4-core box is a <i>throttle</i>

This distinction caused a wrong recommendation once in this project's history: `2` is the Helm answer, given for a compose box. On compose, run `nproc` and decide from that.

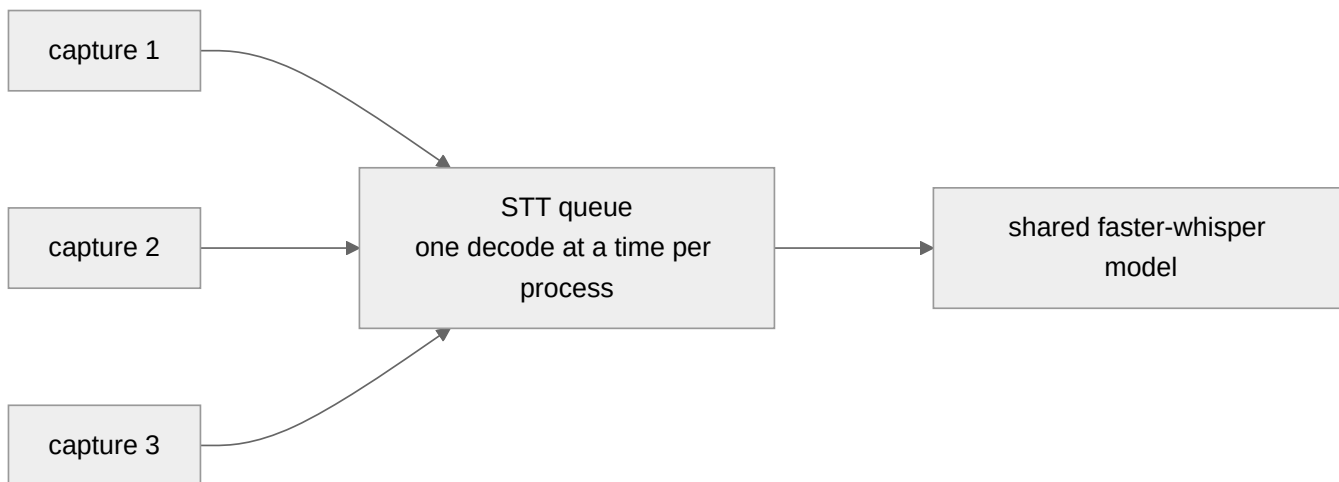
Tuning accuracy vs speed

Knob	Effect on accuracy	Effect on speed
<code>STT_MODEL_SIZE</code> (<code>tiny</code> → <code>small</code> → <code>medium</code> → <code>large-v3</code>)	Large effect	Large cost — must also be baked into the image
<code>STT_BEAM_SIZE</code> (default 5)	Moderate	Roughly linear
<code>STT_COMPUTE_TYPE</code> (<code>int8</code> , <code>int8_float16</code> , <code>float16</code>)	Small	Noticeable
<code>STT_VAD_FILTER</code>	Removes silence — usually helps	Helps
<code>STT_CPU_THREADS</code>	None	Real, when matched to the actual quota

If transcription is too slow, reach for `cpu_threads` and `vad_filter` **before** dropping the model size. Only the last one costs you accuracy.

7. Concurrency and sizing

Because a decode is serialised per process and CPU-bound, concurrent captures queue.



Rules of thumb for the single-VM deployment:

Cores	Sensible <code>STT_CPU_THREADS</code>	Comfortable concurrent captures
2	<code>2</code>	1–2
4	<code>0</code> or <code>4</code>	2–3
8	<code>0</code>	4–6

"Comfortable" means the third user is not waiting past the 300-second browser budget. Empirically, three concurrent two-minute captures on the current production box all completed successfully.

To scale beyond that, **add STT replicas** — the service is stateless and the model is in the image. It is the one component in PRISM where more replicas directly buys more throughput.

8. Storage, retention and secrets

Thing	Where	Notes
Audio bytes	MinIO, bucket <code>prism-vocx-captures</code>	Same platform rule as Register documents. The <code>s3://bucket/key</code> reference rides on the committed interaction
Playback URLs	presigned against <code>VOCX_S3_PUBLIC_ENDPOINT_URL</code>	Signed for the browser's view of MinIO, not the container's
Retention	<code>VOCX_AUDIO_RETENTION_DAYS</code> (default <code>0</code> = keep)	
Local state	<code>vocx_state:/data/vocx</code>	Drafts, PKCE verifiers
Google OAuth client	<code>deploy/vocx-secrets/client_secret.json</code> , mounted read-only	git-ignored . Absent = Google integration off; register writes still work
Per-user Google tokens	<code>VOCX_TOKENS_DIR=/data/vocx</code>	Never in the repo or a delivery archive

`deploy/vocx-secrets/` is one of the three paths that must never enter git or a zip. See [09 Backup & restore](#) §2 for the verification command.

9. Google Workspace integration

Per-user OAuth, so notes are written as the RM, not as a service account.

Redirect URI (must match Google Cloud Console exactly):
`https://<host>/vocx/v1/auth/callback`

Set as `VOCX_OAUTH_REDIRECT_URI` — **as the browser reaches it**, through the edge. This is the commonest misconfiguration: an internal URL here produces a `redirect_uri_mismatch` at Google.

`/vocx/v1/auth/callback` is in `GATEWAY_AUTH_EXEMPT_PATHS` — Google calls it without a PRISM bearer.

If the Google integration is off or fails, **the capture still completes**: the Register write is the part that matters, and the Drive/Docs write is skipped.

10. Intelligence switches

Each independently switchable, all defaulting **on**:

Variable	What it does
<code>VOCX_STT_PRIMING</code>	Primes Whisper with expected vocabulary — client names, finance terms — so "Mecpower" is not heard as "make power"
<code>VOCX_EXTRACT_GLOSSARY</code>	A domain glossary for the extraction step
<code>VOCX_EXTRACT_FEW_SHOT</code>	Few-shot examples for extraction
<code>VOCX_EXTRACT_STRUCTURED</code>	Structured (rather than free-form) extraction output

`VOCX_ANTHROPIC_API_KEY` empty $\Rightarrow$ the **offline stub** runs. Typed transcripts still work end to end, which is what makes dev and CI able to exercise the whole lifecycle without a vendor account.

11. Configuration reference (VocX)

Variable	Default (compose)	Meaning
<code>VOCX_REGISTER_BASE_URL</code>	<code>http://register:8000</code>	The book
<code>VOCX_REGISTER_API_KEY</code>	<code>\${SVC_VOX_KEY}</code>	The named <code>svc_vox</code> principal
<code>VOCX_API_KEYS</code>	<code>\${VOCX_FRONT_KEY}</code>	VocX's own front door — only the gateway's injected key is accepted
<code>VOCX_INTERNAL_SIGNING_SECRET</code>	shared	Verify + re-mint the caller's context so the human authorises Register writes
<code>VOCX_ORCHESTRATOR_URL / _API_KEY</code>	orchestrator	Company-name captures with no resolved subject run as durable workflows
<code>VOCX_STT_BACKEND</code>	<code>api</code>	<code>api</code> (the STT service) · <code>faster_whisper</code> (in-process fallback) · <code>stub</code>
<code>VOCX_STT_API_URL</code>	<code>http://stt:8000/v1/audio/transcriptions</code>	
<code>VOCX_STT_API_KEY</code>	<code>\${STT_API_KEY}</code>	
<code>VOCX_S3_*</code>	MinIO	Endpoint, public endpoint, bucket, credentials
<code>VOCX_AUDIO_RETENTION_DAYS</code>	<code>0</code>	<code>0</code> = keep forever
<code>VOCX_GOOGLE_CLIENT_SECRET_FILE</code>	<code>/run/vocx-secrets/client_secret.json</code>	Absent = Google off
<code>VOCX_OAUTH_REDIRECT_URI</code>	—	As the browser sees it
<code>VOCX_TOKENS_DIR</code>	<code>/data/vocx</code>	Per-user Google tokens
<code>VOCX_DEV_UI</code>	<code>true</code> in dev	The <code>/vocx/v1/dev-ui</code> console. Pinned off by the prod-posture overlay

12. Troubleshooting

Symptom	Cause	Fix
"VocX did not answer in time"	STT exceeded the 240 s budget — usually queued behind other captures	Check <code>docker compose logs stt</code> ; raise <code>STT_CPU_THREADS</code> to the real quota; add an STT replica
Stage strip reverts to blank mid-capture	Historically: the patience window started before the upload finished	Fixed in <code>RecordTab.tsx</code> . If it recurs, check <code>captureStatus</code> polling
Recording stops at 3 minutes	Working as designed — <code>MAX_SECONDS</code>	Raise <code>VITE_VOCX_MAX_SECONDS</code> , rebuild <code>ui</code> , reload nginx, hard-refresh
413 on capture	Clip exceeds <code>max_audio_bytes</code> (25 MB) or nginx's 64 m	Shorten the clip, or raise both
Transcript quality poor	Model size, or priming off	Check <code>STT_MODEL_SIZE</code> and <code>VOCX_STT_PRIMING</code> . Do not touch <code>cpu_threads</code> — it changes nothing about accuracy
Company matched to the wrong entity	Name similarity	The approval card exists for this. Confirm the company; consider an alias
Google notes not written	Client secret absent, or <code>redirect_uri_mismatch</code>	Check the mount and that <code>VOCX_OAUTH_REDIRECT_URI</code> matches Google Console exactly
STT will not start	Baked model does not match <code>STT_MODEL_SIZE</code>	Deliberate fail-fast — rebuild the image with the matching <code>ARG STT_MODEL_SIZE</code>
Capture succeeded, no interaction on the timeline	The gate raised an approval card	Open the VocX panel; the draft is waiting